CLAUDE CERTIFIED ARCHITECT — FOUNDATIONS

Objective 11: CI/CD CLI Invocations

> **Exam objective:** Configure Claude Code CLI invocations for automated CI/CD pipelines using non-interactive flags, permission modes, and cost and turn limits that prevent runaway executions.

Q1.

A release pipeline runs nightly in GitHub Actions. The job sometimes hangs because Claude Code waits for user approval before reading changed files. The team wants fully unattended execution while still preventing arbitrary writes outside the workspace. Which configuration is best?

A. Run Claude Code interactively with default permissions so the model can decide when to ask B. Use non-interactive execution with a workspace-scoped permission mode that allows needed reads and controlled writes C. Disable all tool permissions and rely on the prompt alone D. Increase the timeout to several hours so a human can approve requests later

Answer: B **Explanation:** Automated CI/CD requires non-interactive mode so the run cannot block on prompts. A workspace-scoped permission mode lets the agent access the repository and make intended edits without granting unnecessary broader access. Interactive defaults and human intervention defeat unattended execution, while disabling all tools prevents useful work.

Q2.

A team wants Claude Code to review a pull request in CI, but they are worried about runaway loops that repeatedly re-read files and call tools. Which pair of controls most directly addresses that risk?

A. Longer job timeout and broader permissions B. Higher model context window and verbose logging C. Cost limit and turn limit D. Retry count and branch protection

Answer: C **Explanation:** Cost limits cap spend and turn limits cap the number of agent interaction cycles, together constraining runaway behavior. Job timeouts may stop a job eventually but do not directly govern model/tool usage. Context size and logging help observability, not execution bounds.

Q3.

Your pipeline step is intended only to analyze changed Terraform files and produce a machine-readable summary. Which invocation design is most appropriate?

A. Interactive mode with unrestricted shell access so the agent can explore the environment freely B. Non-interactive mode, minimal read permissions for the repo, structured output instructions, and explicit execution limits C. Local desktop mode because it provides richer prompts D. A configuration that omits limits so the agent can reason as long as needed

Answer: B **Explanation:** CI analysis jobs should be deterministic, bounded, and minimally privileged. Non-interactive mode prevents prompts, least-privilege read access limits exposure, and explicit output formatting plus turn/cost ceilings makes the step suitable for automation. Unrestricted or unbounded configurations increase risk without benefit.

Q4.

An organization mandates that CI jobs must fail quickly rather than continue after the model reaches its budget. What is the best reason to set a strict cost ceiling even for small repositories?

A. It guarantees the model will always choose the cheapest tools B. It enforces a hard stop if execution starts to expand unexpectedly C. It improves answer quality by shortening prompts D. It replaces the need for repository permissions

Answer: B **Explanation:** A strict cost ceiling acts as a hard guardrail against unexpected expansion in tool use or reasoning depth. It does not optimize tool selection automatically, improve quality directly, or substitute for permission controls. In CI, predictable failure is often preferable to silent overconsumption.

Q5.

A build engineer configures Claude Code in a merge-validation pipeline. The model only needs to read the checked-out repository and emit JSON to stdout. Which permission posture is best?

A. Allow repository reads and no unnecessary external tool access beyond what is required to inspect files and print results B. Grant network, shell, and filesystem access across the whole machine to maximize flexibility C. Permit write access to parent directories in case the agent needs scratch space D. Enable interactive approvals so the agent can escalate if needed

Answer: A **Explanation:** Least privilege is the correct approach when the task is limited to repository inspection and emitting results. Broader machine, network, or parent-directory access expands risk without helping the stated objective. Interactive escalation is incompatible with unattended CI.

Q6.

A team notices that a Claude Code CI step occasionally stops mid-task because it hits the turn limit. The run is not runaway; it simply needs to inspect more files for large monorepo changes. What is the best adjustment?

A. Remove the turn limit entirely B. Increase the turn limit modestly while keeping the cost ceiling and permission scope intact C. Broaden tool permissions to compensate D. Switch to interactive mode so a user can resume the run

Answer: B **Explanation:** If legitimate work exceeds the current turn budget, the right fix is to tune the limit upward conservatively while preserving other safeguards. Removing the limit eliminates an important guardrail. Permission changes and interactive mode do not solve boundedness appropriately.

Q7.

You are designing a CI step that patches formatting issues automatically and commits the result in a temporary branch. Which additional safeguard is most important beyond non-interactive mode?

A. A permission mode that narrowly allows only the workspace writes required for the formatting task B. Unlimited turns so the model can retry until formatting succeeds C. Full network access to search for style guides D. Permission to modify the CI runner's global configuration

Answer: A **Explanation:** Automatic fix-up steps need carefully scoped write permissions limited to the repository workspace. Unlimited retries invite runaway behavior, and external network or runner-configuration access is unnecessary for formatting changes. Least-privilege write scope keeps automation safer.

Q8.

A platform team wants each CI invocation to behave predictably across repositories. Which practice best supports that goal?

A. Let each repository omit limits so the model can adapt freely B. Standardize non-interactive flags, permission modes, and budget caps in reusable pipeline templates C. Require engineers to monitor every run manually D. Depend on model defaults because they are always stable

Answer: B **Explanation:** Reusable pipeline templates enforce consistent invocation behavior across projects. Standardizing execution mode, permission posture, and resource limits reduces drift and operational surprises. Manual monitoring and implicit defaults are less reliable for repeatable CI.

Q9.

A security reviewer asks why a Claude Code pipeline step should explicitly disable approval prompts rather than relying on the fact that no person is present to respond. What is the best answer?

A. Because explicit non-interactive configuration prevents the tool from waiting indefinitely for approvals that will never arrive B. Because disabling prompts automatically grants broader permissions C. Because approval prompts only exist on local machines, not CI D. Because non-interactive mode eliminates the need for cost controls

Answer: A **Explanation:** In unattended environments, waiting for approval is a failure mode; explicit non-interactive configuration removes that ambiguity. It does not broaden permissions, and approval behavior is still relevant in CI-capable tools. Cost controls remain independently necessary.

Q10.

A repository contains generated files that can trigger huge diff reviews. The team wants Claude Code to stop before spending heavily if review scope explodes. Which control is most directly aligned to that requirement?

A. Set a maximum cost budget per invocation B. Increase shell privileges so the model can filter files faster C. Remove turn limits to avoid partial output D. Run the job only on weekdays

Answer: A **Explanation:** A maximum cost budget directly limits spend when scope expands unexpectedly, such as when large generated diffs appear. Broader privileges do not control expenditure, and removing turn limits makes runaway behavior worse. Schedule restrictions are unrelated.

Q11.

A DevOps engineer is choosing between a single generous limit and layered safeguards. Which approach best reflects good Claude Code CI/CD design?

A. Use only a long external job timeout and no Claude-specific limits B. Combine non-interactive mode, least-privilege permissions, turn limits, and cost limits C. Allow all tools but depend on prompt wording to prevent misuse D. Increase retries until failures disappear

Answer: B **Explanation:** Layered controls are the strongest pattern for safe automation. Claude-specific limits constrain agent behavior directly, while least-privilege permissions reduce blast radius and non-interactive mode ensures unattended operation. Timeouts and prompt-only controls are weaker and less targeted.

Q12.

A pipeline step must summarize lint failures after running existing repository scripts. The agent needs shell access only to execute the known lint command and read its output. What is the best setup?

A. Grant unrestricted shell access for future flexibility B. Use non-interactive mode with tightly constrained shell/tool access for the specific workflow and explicit turn/cost ceilings C. Deny shell access and ask the model to infer lint failures from source code alone D. Allow network access so the model can compare errors with online examples

Answer: B **Explanation:** The task requires bounded automation and only the minimum shell capability needed to run known project commands. Unrestricted shell access and network access exceed the requirement, while removing shell access makes the workflow incomplete. Explicit ceilings protect the CI environment.

Q13.

An enterprise team wants pipeline failures to be easy to diagnose when Claude Code stops early. Which design choice best supports that without weakening safeguards?

A. Remove cost and turn limits so failures are less likely B. Keep limits in place and ensure the step emits clear status/output indicating whether a guardrail was hit C. Switch every job to interactive mode for live debugging D. Grant admin permissions so the model can self-remediate

Answer: B **Explanation:** Good automation preserves safety controls while making failure causes observable. Clear logging or structured output about hitting a budget or turn cap supports

diagnosis without removing guardrails. Interactive debugging and admin permissions are unnecessary for routine CI operation.

Q14.

A team is migrating from ad hoc local use of Claude Code to a production deployment pipeline. Which invocation change is most essential?

A. Preserve the interactive approval flow because it worked locally B. Move to explicit non-interactive execution with policy-defined permissions and execution limits C. Remove all restrictions to avoid surprising production failures D. Depend on engineers to watch logs and cancel bad runs manually

Answer: B **Explanation:** Production pipelines need explicit, unattended, policy-driven behavior. Non-interactive execution and codified permissions and budgets make runs reproducible and safe. Local interactive habits do not translate well to automated environments.

Q15.

A CI workflow uses Claude Code to generate a changelog draft. The team wants to guarantee the process cannot continue indefinitely even if the prompt is ambiguous. Which configuration is the clearest protection?

A. A turn limit that caps the number of agent cycles, ideally paired with a cost limit B. A larger model so it finishes in one pass C. More verbose instructions without any execution limits D. Full filesystem access so the agent can gather more context

Answer: A **Explanation:** Ambiguous prompts can cause repeated exploration, so a turn limit provides a direct bound on execution length, and a cost limit complements it with a spend cap. Larger models or broader access may still loop. Clearer instructions help, but they are not a hard stop.